

RemSolution — Plan de développement

Rédigé contre le commit `139750e` · confronté à votre liste de points · ce qui reste, ce qui doit être amélioré, les problèmes, et les estimations.

Mise à jour du 2026-09-02 — relecture du code, pas des tableaux. Les lots 1, 2 et 3 sont livrés, ainsi que la sécurité, la CI et une grande partie de l'addendum. Les statuts d'origine sont conservés tels quels ci-dessous pour garder la trace de ce qui a été décidé et pourquoi ; **l'état courant est celui du §0.1**, et c'est le seul tableau à lire pour savoir ce qu'il reste.

0. Résumé exécutif

La majorité de votre liste est déjà développée. En confrontant vos points au code, la répartition est la suivante :

	Points
✅ Déjà fait (et souvent au-delà de ce que la note demandait)	34
🟡 Partiel — la base existe, il manque une partie	9
❌ À faire	14
🔴 Problème / dette à corriger	7

Reste à faire, hors mobile et GPS : 38,5 jours ≈ 8 semaines (détail chiffré en §5). Avec le mobile (deux applications) et le GPS : ~20 semaines.

Trois points bloquent la mise en production et doivent passer en premier (détaillés en §4) : la création automatique de l'administrateur d'agence, les sauvegardes, et le déploiement des migrations. Aucun des trois n'est gros — environ 4 jours au total — mais aucun ne peut être différé.

0.1 État vérifié au 2026-09-02

Chaque ligne a été vérifiée dans le code, pas dans un tableau de suivi.

Livré depuis la rédaction du plan

Réf	Point	Où le voir
4.1	Administrateur d'agence créé avec l'agence	<code>CreateAgencyCommand.cs</code> (<code>AdminTemporaryPassword</code> , mot de passe temporaire + <code>PasswordChangeRequiredMiddleware</code>)

Réf	Point	Où le voir
4.2	Migrations appliquées au déploiement, plus au démarrage	<code>.github/workflows/azure-dev.yml</code> (<code>dotnet ef migrations bundle</code> puis <code>efbundle</code>) ; le démarrage ne migre plus qu'en dev (<code>Database:MigrateOnStartup</code>)
4.3	Sauvegardes	<code>infra/core/database/sqlserver/sqlserver.bicep</code> : point-in-time 35 j + LTR hebdo/mensuelle/annuelle ; <code>scripts/restore-drill.ps1</code> pour la restauration
4.4	<code>Renting</code> en agrégat riche	<code>Renting.Create/Start/Complete/Cancel</code> , setters privés, événements
4.5	Verrou par véhicule	<code>AcquireCarWriteLockAsync</code> , le verrou agence restant pour les quotas
4.7	Nettoyage	<code>cookies_*.txt</code> , <code>info.docx</code> , <code>Colour.cs</code> , <code>UnsupportedColourException.cs</code> supprimés
2.1	Tests	829 tests : 111 domaine, 159 application, 24 infrastructure, 535 fonctionnels ; <code>Web.AcceptanceTests</code> a Login + Cars
2.2	Assistant de création de location	<code>renting-form.component.ts</code> — stepper 3 étapes (véhicule et dates → client → paperasse)
2.5	CI	<code>.github/workflows/ci.yml</code> : build, tests unitaires, build Angular, conventions front, fonctionnels + intégration
2.6	Durcissement sécurité	limitation de débit (<code>src/Web/Infrastructure/RateLimiting.cs</code>), durcissement des téléversements, fichiers de débogage supprimés
3.1	Seuils de dépense par voiture	<code>CarExpenseSchedule</code>
3.2	Frais supplémentaires au retour	<code>RentingFee</code>

L'addendum est livré à l'exception de A.6 et A.7 — voir son propre encadré.

Ce qu'il reste

Réf	Point	État	Effort
2.9 / 3.6	Export des statistiques	❌ rien dans <code>Features/Statistics</code> , aucune bibliothèque tableur référencée	1 j
3.6	Purge des données personnelles / droit à l'effacement	❌ aucun job de purge, aucune anonymisation	1,5 j
A.6	Alertes de supervision	❌ aucune règle d'alerte dans <code>infra/</code>	1,5 j
3.6	Signalements (réclamations) + triage plateforme	❌ absent	1 j

Réf	Point	État	Effort
3.6	Test de charge du chemin de recherche	✗ <code>loadtesting.bicep</code> existe (gabarit) mais n'est pas référencé par <code>main.bicep</code>	1 j
2.7	Signature : libellé à corriger ou vraie signature	✗ <code>fr.json</code> annonce toujours « Contrats + signature électronique » alors que le code ne pose que des lignes manuscrites	0,5 j ou 6 j
2.4	Hébergement réellement déployé et vérifié	● le Bicep est complet (Key Vault, sauvegardes) ; le déploiement effectué ne se lit pas dans le dépôt	3 j
4.3	Restauration réellement exécutée et chronométrée	● le script existe, l'exécution documentée non	0,5 j
2.8	Inscription libre-service des agences	✗ la création d'agence reste réservée à l'administrateur plateforme	hors dev. + 3 j

Et dix points ajoutés le 2026-09-02 — papiers, devises, parcours de réservation, fiabilité, assistant d'ouverture d'agence : voir §8. Cinq sont livrés (N.9, N.1, N.5, N.6, N.7) ; il en reste ≈ 6,5 j.

À noter : les **signalements** de la ligne §3.6 ci-dessus sont désormais liés à N.8 — c'est la même fonctionnalité vue des deux côtés, et le plan est de les construire ensemble.

Écarté volontairement, avec la raison

Réf	Point	Pourquoi
4.6	Outbox transactionnel	Sa justification est l'atomicité du paiement en ligne et des notifications push, tous deux hors périmètre. Y faire passer le courriel a été envisagé puis écarté : toute file durable persiste le corps du message, or le courriel d'accueil porte un mot de passe temporaire. À revoir le jour où un prestataire de paiement ou un webhook a réellement besoin de cette atomicité.
2.3	Paiement en ligne	Hors périmètre : demande un prestataire (Paymee / Konnect / Flouci), un compte marchand et des identifiants de bac à sable.
A.7	Prestataire d'envoi de courriels	Demande un compte chez un prestataire ; <code>SmtpEmailSender</code> reste en place d'ici là.
3.3–3.5	Mobile agence, mobile client, GPS	Décision produit non prise (voir §6).
A.10	Second conducteur / assurance	Tranché et implémenté : le contrôle de permis périmé s'applique au locataire <i>et</i> au second conducteur, bloquant sauf acquittement explicite (<code>CreateRentingCommand</code>).

1. Ce qui est déjà fait

À noter avant de lire : plusieurs points de votre liste sont non seulement faits, mais implémentés plus finement que la note ne le demandait. Inutile de les replanifier.

Administrateur système

- CRUD types de dépenses, marques, modèles, types de services additionnels — **fait**
- CRUD agences — **fait** (avec pin sur carte, devise, succursales créées dans la même transaction)
- CRUD utilisateurs, avec attribution d' `AgencyId` — **fait** (`CreateAgencyUserByAdminCommand` , `ResetAgencyUserPasswordCommand` , activation/désactivation)
- Abonnements d'agences + CRUD formules — **fait** (quotas voitures/clients/utilisateurs appliqués sous verrou, statuts Actif/Suspendu/Expiré)
- Tableau de bord : toutes agences / abonnements / voitures et clients par pays — **fait**, et plus complet que la note : agences au quota, abonnements arrivant à échéance, agences sans abonnement, répartition par formule, recettes des formules actives
- « Toutes les données de chaque agence + possibilité de tout faire pour chacune » — **fait** via l'entrée en espace de travail d'agence (`PlatformAdminImpersonationMiddleware`), chaque entrée étant auditée
- Import de modèles de contrat/facture par défaut — **fait** (`ImportDocumentTemplateCommand` , `DocumentTemplateExamples` avec contrat et facture standard, localisés)

Administrateur / utilisateurs d'agence

- CRUD utilisateurs avec attribution d'autorisations — **fait** (écran Équipe ; n'offre que les autorisations des modules activés)
- Configuration : types de dépenses, types de services additionnels, choix du modèle de facture — **fait** (`SetDefaultDocumentTemplateCommand`)
- CRUD voitures, clients, réservations, locations, paiements — **fait**
- Modèle de contrat avec affectation automatique des champs, et manuelle quand le champ n'est pas clair — **fait**, exactement comme demandé : liaison automatique quand le nom de la variable est un chemin connu, sinon *valeur fixe, demander à chaque fois* ou *vide*
- Ajout d'une location complète (client existant ou nouveau, voiture, services additionnels, facture, paiements, contrat) — **fait**
- Ajout de dépenses (voiture / type / montant) — **fait**
- Crédits (clients : montant location/payé ; dépenses : montant/payé) — **fait**, et l'interface Dépenses a bien été fusionnée dans Paiements comme vous le demandiez
- Tableau de bord clients/argent par mois, année ou date choisie — **fait**
- Demandes de réservation (accepter / refuser avec motif) — **fait**, motif obligatoire et visible du client
- Messagerie avec le client en cours de location — **fait**, et fermée automatiquement quand la location est clôturée (`ChatMessage.CanPostTo`)
- Widgets d'accueil au choix de l'utilisateur, limités à ses autorisations — **fait**
- Succursales avec adresse choisie sur la carte — **fait**

- Calendrier — **fait** (`GetBookingCalendarQuery` , disponible comme panneau d'accueil)
- Notifications / e-mails (entretien voitures, locations en retard, réservations à venir, pour le personnel et les clients concernés) — **fait**, 6 types, balayage horaire par Hangfire
- Statistiques par mois/année, avec filtre voiture ou toutes voitures — **fait** (granularité mois/année, filtre `CarId` , accès depuis la fiche voiture)
- Depuis l'accueil : accès direct aux réservations à valider, locations en retard, réservations du jour — **fait** (écran Accueil unifié en un seul appel)
- Recadrage de la photo depuis la CIN, affichée dans la liste clients — **fait** (`SkiaPortraitCropper` , détection de peau en YCbCr, repli sur l'emplacement conventionnel)
- Voitures avec photos en liste et en détail — **fait** (original + vignette + moyenne, générées en arrière-plan)
- Fiche voiture : dépenses liées groupées par type — **fait**
- Caution — **fait** (`DepositAmount` sur location et réservation, reportée à la conversion)
- Liste clients/voitures : historique des locations, crédits, louer directement depuis la liste, retour si loué, statut — **fait**
- Paiement direct + justificatif joint au paiement — **fait** (`AddPaymentProofFile`)
- Client : email + regroupement avec les comptes utilisateurs, invitation à l'espace client — **fait** (`InviteClientCommand` , envoi automatique sur création/modification/location)
- Client : notation après la location — **fait** (une note par location terminée, sur la voiture et/ou l'agence)
- Client : choix pays/lieu + dates, vue agence avec sa note et ses voitures — **fait** (+ vue carte avec « chercher dans cette zone »)
- Prolongation de la date de fin sans historique séparé — **fait** (`ChangeRentingEndDateCommand` , retarification)
- Modules et autorisations liés (features → permissions) — **fait**, 16 modules, `FeatureCatalog` fait l'intersection
- « Réservation : pourquoi les actions de l'agence sont dans l'entité » — **résolu** : `Reservation` est un vrai agrégat avec machine à états gardée, setters privés et événements de domaine ; l'identité de l'appelant est vérifiée par la politique d'autorisation, pas par la structure de l'entité

2. Partiel — la base existe, il manque une partie

2.1 Tests — **livré (2026-09-02)** · *diagnostic d'origine ci-dessous*

619 tests existent : 479 fonctionnels, 119 applicatifs, **21 seulement au niveau domaine**. Les projets `Infrastructure.IntegrationTests` et `Web.AcceptanceTests` sont des coquilles vides (9 fichiers, 0 test).

Le déséquilibre n'est pas un oubli de rigueur : il n'y a presque rien à tester au niveau domaine parce que `Renting` est une entité anémique (voir §4.4). Rendre `Renting` riche fait apparaître les tests domaine naturellement.

- Tests domaine après enrichissement de `Renting` : **2 j**

- Un parcours E2E fiable (SpecFlow + Playwright déjà câblés) : **1,5 j**
- Tests d'intégration infrastructure (stockage fichiers, imagerie, jobs) : **1,5 j**

2.2 Interface de location — ● (fonctionnelle, à retravailler) · 3 j

Vous notiez : « *interface de location pas claire et pas belle, le prix devrait être calculé automatiquement par date mais modifiable, un assistant serait peut-être mieux* ».

Le calcul automatique existe (`IPricingService` calcule depuis les dates et le tarif journalier, le prix est figé à la réservation et modifiable). L'assistant, non : `renting-form.component` reste un formulaire unique. À faire : un assistant en étapes (client → voiture et dates → services additionnels → prix → confirmation).

2.3 Paiement en ligne — ● (structure prête, aucun prestataire) · 5 j

Le module `OnlinePayment` existe, l'état `Payée` existe, `PaymentMethod` couvre espèces/carte/virement/chèque. **Aucun prestataire n'est intégré** — pas de Stripe, Paymee, Konnect ou Flouci dans le code. Il faut choisir un prestataire (pour la Tunisie : Paymee ou Konnect), gérer le retour de paiement et les webhooks (voir §4.3 : l'Outbox devient nécessaire ici).

2.4 Hébergement — ● (Bicep existe, non déployé) · 3 j

`infra/` contient des modèles Bicep et `azure.yaml` pour Azure Developer CLI, et un workflow `azure-dev.yml`. Ce qui manque : un déploiement réellement effectué et vérifié, les secrets en Key Vault (l'abstraction existe), le nom de domaine, les certificats.

2.5 CI/CD — ● · 1,5 j

Un seul workflow GitHub (`azure-dev.yml`, déploiement). Il manque : compilation + tests sur chaque PR, tests fonctionnels sur la branche principale, et surtout **l'application des migrations au déploiement** (voir §4.2).

2.6 Sécurité — ● (les fondations sont bonnes, il reste du nettoyage) · 2 j

Solide : isolation par locataire vérifiée par test de convention, autorisations par politique, jetons courts + rafraîchissement, verrou applicatif, RowVersion, audit.

À corriger : mot de passe d'amorçage en dur (`"PlatformAdmin1!"`), fichiers de débogage commités (`cookies_reg.txt`, `info.docx` à la racine, trois `cookies_*.txt` dans `ClientApp`), limitation de débit et durcissement des points d'entrée publics (téléversements : taille, type réel, extensions), scan de secrets sur l'historique complet du dépôt.

2.7 Signature électronique — ● (à clarifier) · 6 j si réelle

Le libellé français du module dit « Contrats + signature électronique », mais le code ne fait que des **lignes réglées pour signature manuscrite** (`DocumentBlockType.Signatures`). Deux options :

- **Option A — corriger le libellé** (0,2 j). Si la signature papier au comptoir suffit, c'est le choix honnête.
- **Option B — vraie signature électronique** (6 j) : capture de la signature sur écran/tablette, apposition dans le PDF, horodatage, preuve de consentement. Attention : la valeur juridique en Tunisie relève de

l'ANCE ; une signature « dessinée » n'est pas une signature électronique qualifiée. À arbitrer avec un juriste avant de développer.

2.8 Commercialisation — ● · hors développement

Rien dans le code, et c'est normal : site vitrine, tarification publique, inscription des agences en libre-service, facturation des abonnements. À noter : **l'inscription en libre-service d'une agence est impossible tant que §4.1 n'est pas corrigé.**

2.9 Statistiques — ● (l'essentiel est là) · 1 j

Il manque l'export (Excel/PDF), régulièrement demandé dès qu'un gérant veut transmettre ses chiffres à son comptable.

3. À faire

3.1 Seuils de dépense par voiture — ✗ · 2,5 j

Votre point : « *depuis le type de dépense, je veux lier ceux à notifier avec les voitures, en création/modification/détail voiture, et le paramètre doit être modifiable. Par exemple : vidange liée à la voiture 15412154 tous les 8000 km et à la voiture 1254548 tous les 10000 km.* »

Aujourd'hui le seuil est porté par `ExpenseType` **seul** (`AfterMonth` / `AfterKilometer`), donc global à l'agence : une vidange tous les 8000 km pour tout le monde. Un utilitaire ne s'entretient pas comme une citadine — c'est une vraie limite.

À développer : une table de liaison `CarExpenseSchedule(CarId, ExpenseTypeId, AfterMonth?, AfterKilometer?)`, le seuil du type servant de valeur par défaut quand aucune ligne n'existe. Puis le calculateur d'échéances doit consulter la surcharge, et l'interface doit permettre l'édition depuis la fiche voiture **et** depuis le type de dépense (les deux sens que vous décrivez).

3.2 Frais supplémentaires au retour — ✗ · 2 j

Votre point : « *au retour de location, on peut ajouter de l'argent (retard, réparation, kilométrage supplémentaire...)* ».

La boîte de dialogue de retour ne comporte que le kilométrage et la date. On peut contourner par un service additionnel, mais ce n'est pas la bonne place : ce sont des frais constatés au retour, pas des options vendues au départ. À développer : des lignes de frais typées (retard / dommage / kilométrage excédentaire / carburant / nettoyage) saisies dans la boîte de retour et intégrées au prix final et à la facture.

3.3 Application mobile administrateur/agence — ✗ · 5 semaines

Rien n'existe. Décision à prendre : application unique multi-rôles ou deux applications ; et technologie (le plus cohérent avec une équipe .NET : .NET MAUI ou Blazor Hybrid ; le plus rapide à livrer si l'équipe est plutôt web : Ionic/Capacitor par-dessus l'Angular existant, ce qui permet de réutiliser le client NSwag).

Inclut l'OCR des documents pour préremplir le contrat, prévu de longue date — les originaux sont déjà conservés en pleine résolution précisément pour cela.

3.4 Application mobile client — ✖ · 4 semaines

Recherche, réservation, téléversement des documents, messagerie, notifications *push*. À noter : les notifications *push* n'existent nulle part aujourd'hui (tout passe par courriel et notifications dans l'application) — il faut donc ajouter Firebase ou équivalent, ce qui se greffe naturellement sur l'Outbox (\$4.3).

3.5 Extension GPS — ✖ · 3 semaines

Rien n'existe côté suivi. `NetTopologySuite` est déjà en place pour la géographie (succursales, recherche par distance), ce qui aide, mais le suivi de véhicule est un autre métier : choix du boîtier, ingestion des positions, historique des trajets, géorepérage, et une décision de volumétrie (une position par minute et par véhicule, c'est des millions de lignes — cela ne va pas dans la base transactionnelle sans réflexion).

3.6 Divers — ✖

- **Purge des données personnelles et droit à l'effacement** (obligation légale dès qu'on détient des passeports) : politique de rétention, job Hangfire de purge, anonymisation du client en conservant les lignes financières — **1,5 j**
- **Test de charge du chemin de recherche** au volume cible (2 000 agences / 100 000 voitures) pour valider les plans d'exécution des anti-jointures et de la recherche spatiale — **1 j**
- **Signalements (réclamations)** avec écran de triage pour l'administrateur plateforme — **1 j**
- **Export des statistiques** (voir §2.9) — **1 j**

4. Problèmes — par ordre de gravité

4.1 ● BLOQUANT — Aucun administrateur n'est créé avec l'agence

Votre note le dit : « *quand on ajoute une agence, un admin d'agence est créé automatiquement* ». **Ce n'est pas le cas.** `CreateAgencyCommandHandler` crée l'agence, ses réglages et ses succursales, mais **aucune commande du système n'attribue le rôle `AgencyAdministrator`**. Une agence créée n'a personne qui puisse s'y connecter.

Conséquences : impossible d'ouvrir une agence pour un vrai client sans intervention manuelle en base ; impossible d'ouvrir l'inscription en libre-service.

À faire : créer l'utilisateur administrateur dans la même transaction que l'agence, avec un mot de passe temporaire, l'obligation de le changer à la première connexion (le middleware

`PasswordChangeRequiredMiddleware` existe déjà) et l'envoi du courriel d'accueil. **Estimation : 1 j**

4.2 ● BLOQUANT — Migrations appliquées au démarrage de l'application

`ApplicationDbContextInitialiser.InitialiseAsync` appelle `MigrateAsync()` au démarrage. Sur une seule instance, cela fonctionne. **Dès la deuxième instance (ou un redémarrage pendant un déploiement), deux processus appliquent les migrations en même temps** — et le résultat va de l'erreur bruyante à un schéma à moitié migré.

Aggravant en multi-locataire : ce n'est pas un client qui tombe, c'est tout le monde.

À faire : générer un *bundle* de migrations (`dotnet ef migrations bundle`) appliqué à l'étape de déploiement, et retirer l'appel au démarrage. **Estimation : 1 j**

4.3 ● BLOQUANT — Aucune sauvegarde, aucune restauration répétée

Aucune trace de sauvegarde dans `infra/`. Sur une base partagée par toutes les agences, **une base perdue, c'est tous les clients perdus**. Et une sauvegarde jamais restaurée n'est pas une sauvegarde : c'est une hypothèse.

À faire : sauvegardes automatiques SQL Server + stockage des fichiers, plus **une restauration réellement effectuée** dans un environnement propre, documentée, avec le temps qu'elle a pris. **Estimation : 2 j**

4.4 ● Renting est une entité anémique alors que Reservation est un agrégat

`Reservation` a une fabrique, des setters privés, des transitions gardées et des événements. `Renting` a des setters publics, aucune fabrique, aucune machine à états — les invariants ($\text{fin} > \text{début}$, $\text{prix} \geq 0$, kilométrage de retour $\geq$ kilométrage de départ, transitions légales) vivent donc dans les gestionnaires, où ils peuvent être oubliés par le prochain point d'entrée qui touche une location.

C'est aussi la cause directe des 21 tests domaine seulement (§2.1) : il n'y a rien à assérer.

À faire : `Renting.Create(...)`, `Start()`, `Complete(endMileage)`, `Cancel(reason)` avec transitions gardées et événements de domaine ; les gestionnaires se contentent de charger → appeler → sauvegarder. **Estimation : 2,5 j** (+ les 2 j de tests domaine du §2.1)

4.5 ● Le verrou d'écriture est au niveau agence, pas au niveau véhicule

`AcquireTenantWriteLockAsync` verrouille `agency-writes-{agencyId}`. La correction est assurée (c'est un verrou plus large que nécessaire), mais **toutes les écritures d'une agence se sérialisent**, y compris deux réservations sur deux voitures différentes par deux collaborateurs différents.

Invisible à 3 utilisateurs, gênant à 15, bloquant le jour d'un pic. À reprendre en verrou par voiture pour les chemins de disponibilité, en gardant le verrou agence pour ce qui touche réellement les quotas. **Estimation : 1,5 j**

4.6 ● Aucun Outbox — et trois fonctionnalités à venir en ont besoin

Les événements de domaine sont distribués dans la transaction. Correct aujourd'hui. Mais **paiement en ligne (§2.3), notifications push (§3.4) et tout webhook** sont des effets hors processus : sans Outbox, un `commit` suivi d'un plantage perd l'événement silencieusement — un paiement encaissé chez le prestataire et jamais enregistré chez vous.

À faire avant, et non pendant, l'intégration du paiement : table `Outbox` écrite dans la même transaction, distributeur Hangfire. **Estimation : 2 j**

4.7 ● Nettoyage

Mot de passe d'amorçage en dur ; fichiers de débogage commités (`cookies_reg.txt` , `info.docx` , trois `cookies_*.txt`) — à supprimer **et** à vérifier qu'ils ne contiennent pas de jetons de session valides, l'historique Git les conservant ; restes du gabarit (`Colour.cs` , `UnsupportedColourException.cs`) ; `docs/PROJECT_OVERVIEW.md` décrivait un état obsolète (corrigé). **Estimation : 0,5 j**

5. Ordre recommandé et estimations

Lot 1 — Débloquer la production (4 j)

Rien ne peut être livré à un vrai client avant.

	Tâche	Effort
4.1	Création automatique de l'administrateur d'agence	1 j
4.2	Migrations au déploiement (bundle)	1 j
4.3	Sauvegardes + restauration répétée	2 j

Lot 2 — Solidifier le domaine (6 j)

À faire avant d'ajouter des fonctionnalités sur les locations, sinon la dette se démultiplie.

	Tâche	Effort
4.4	<code>Renting</code> en agrégat riche	2,5 j
2.1	Tests domaine	2 j
4.5	Verrou par véhicule	1,5 j

Lot 3 — Vos manques fonctionnels (7,5 j)

Les vrais trous de votre liste.

	Tâche	Effort
3.1	Seuils de dépense par voiture	2,5 j
3.2	Frais supplémentaires au retour	2 j
2.2	Assistant de création de location	3 j

Lot 4 — Exploitation et mise en ligne (8,5 j)

	Tâche	Effort
2.6	Durcissement sécurité + nettoyage (avec 4.7)	2,5 j
2.5	CI/CD complet	1,5 j
2.4	Hébergement déployé et vérifié	3 j
3.6	Purge données personnelles (légal)	1,5 j

Lot 5 — Monétisation (8 j)

	Tâche	Effort
4.6	Outbox (prérequis)	2 j
2.3	Paiement en ligne (prestataire tunisien)	5 j
2.9	Export des statistiques	1 j

Lot 6 — Finitions (4,5 j)

	Tâche	Effort
2.1	E2E + tests d'intégration	3 j
3.6	Test de charge du chemin de recherche	1 j
2.7	Corriger le libellé signature (option A)	0,5 j

Lot 7 — Extensions (optionnel, à décider)

	Tâche	Effort
3.3	Application mobile agence (+ OCR)	5 sem.
3.4	Application mobile client (+ push)	4 sem.
3.5	Extension GPS	3 sem.
2.7	Vraie signature électronique (option B)	6 j

Total

Périmètre	Durée
Lots 1 à 6 — produit vendable et exploitable	38,5 j ≈ 7,7 semaines
+ Lot 7 complet	~20 semaines

Estimations pour un seul développeur à temps plein, hors mobile où une compétence dédiée change fortement la donne.

6. Décisions à arbitrer avant de coder

Cinq points ne sont pas des tâches mais des choix qui déterminent le travail :

1. **Signature électronique** — libellé à corriger, ou vraie fonctionnalité juridiquement valable ? Écart : 0,5 j contre 6 j, plus un avis juridique.
2. **Prestataire de paiement** — Paymee, Konnect, Flouci ? Détermine l'effort réel du §2.3 et si le paiement en ligne est même viable pour vos agences.
3. **Mobile : technologie** — MAUI (cohérent avec .NET) ou Capacitor par-dessus l'Angular existant (bien plus rapide, réutilise le client NSwag) ? Écart possible : plusieurs semaines.
4. **Mobile : une application ou deux ?** Votre liste en prévoit deux. Une seule application multi-rôles coûte moins cher mais mélange deux publics très différents dans les magasins d'applications.
5. **GPS : volumétrie et stockage** — à décider avant la première position écrite, pas après. Une base transactionnelle multi-locataire n'est pas l'endroit pour un flux de télémétrie.

Tranché le 2026-09-02

Question	Décision
Tarif journalier HT ou TTC ?	TTC , sans interrupteur. La facture redescend au HT.
Paieement en ligne	Hors périmètre. Les exigences de paiement sont déclarées et réglées hors ligne, justificatif à l'appui (N.6).
Devises	Conversion à l'affichage seulement (N.4). Une seule devise de facturation par agence.
Frais d'annulation dus par une agence	Pas de flux d'argent : pénalité sur la note de fiabilité + signalement (N.8).
Expiration des papiers	Calculée depuis la date de délivrance , durées réglables par agence (N.1).
Second conducteur / assurance (A.10)	Tranché : le contrôle de permis périmé s'applique aux deux conducteurs, bloquant sauf acquittement.

7. Ce que je recommande de faire cette semaine

Le Lot 1 en entier, dans l'ordre : administrateur d'agence automatique (1 j), migrations au déploiement (1 j), sauvegardes et restauration répétée (2 j).

Ces quatre jours transforment un projet « presque fini mais non livrable » en projet livrable. Tout le reste peut attendre ; ces trois-là, non.

(Lot 1 livré le 2026-09-02 — voir §0.1.)

8. Nouveaux points demandés — 2026-09-02

Dix points ajoutés après relecture du produit. Le tableau donne l'état **vérifié dans le code**, pas l'intention.

Réf	Point	État	Effort
N.9	Chevauchement bloquant à la confirmation	✅ livré — voir ci-dessous	—
N.1	Expiration par défaut des papiers	✅ livré	—
N.5	Réservation en attente + notification	✅ livré	—
N.7	Fiabilité client + frais d'annulation	✅ livré	—
N.8	Annulation par l'agence : note + signalement	❌ absent, dépend des signalements (§3.6)	1,5 j
N.6	Exigences sur réservation confirmée + chat	✅ livré	—
N.4	Devises EUR/TND + taux	❌ absent	2 j
N.10	Assistant d'ouverture d'agence	❌ absent	3 j
N.2	Montants TTC (location, tarif journalier)	✅ livré — décision TTC, <code>TaxBreakdown.FromGross</code> ; reste à écrire « TTC » dans l'interface	0,2 j
N.3	Paramètre TVA	✅ livré — <code>VatRatePercent</code> , <code>TaxIdentifier</code> , <code>FiscalStampAmount</code> dans <code>AgencySettings</code> , exposés dans l'écran Mon agence	—
	Reste		≈ 6,5 j

Ce qui a été livré le 2026-09-02

N.9 — la règle de disponibilité a changé. Une demande `PendingConfirmation` ne bloque plus la voiture ; seules `Confirmed` et `Paid` le font. Le prédicat a donc bougé aux deux endroits qui le portent (`AvailabilityChecker` et `MarketplaceCars.AvailableBetween`), et `ConfirmReservationCommand` prend désormais le verrou par voiture et vérifie la disponibilité avant de confirmer. `BookingConflictException`

transporte le type et l'identifiant de ce qui occupe la période, l'API les expose (`conflictKind` / `conflictId` sur le 409 `booking_conflict`), et le SPA affiche « occupée par la réservation n° 12 ». Conséquences voulues : plusieurs clients peuvent demander la même voiture aux mêmes dates, une location au comptoir reste possible pendant qu'une demande attend, et le marketplace ne cache plus une voiture simplement parce que quelqu'un a posé une question dessus.

N.1 — expiration par défaut. `AgencySettings` porte `CINValidityYears` (10), `PasseportValidityYears` (5) et `DrivingLicenceValidityYears` (10), réglables dans l'écran Mon agence ; 0 = ne se périmé pas ici. `Client.ApplyDefaultDocumentExpiries` remplit une date d'expiration absente depuis la date de délivrance, et les trois chemins qui écrivent les papiers d'un client passent par le même point (`ClientDocumentDefaults`). Une date saisie n'est jamais écrasée. Migration `AddClientDocumentValidityYears` , avec backfill explicite — sans lui, les agences existantes seraient tombées sur 0.

N.5 — la demande entrante est signalée. Nouveau `NotificationKind.ReservationPending` , émis à la création d'une demande (écran agence *et* marketplace) vers les utilisateurs qui détiennent `Reservation.Update` — ceux qui peuvent y répondre — puis une seconde fois par le balayage horaire quand le délai de garde est sur le point d'expirer.

N.7 — fiabilité et frais d'annulation. Note calculée par agence, affichée là où une demande se décide ; seule une annulation faite par le client compte. Politique d'annulation réglable (`CancellationPolicy` : aucun frais / montant fixe / pourcentage, avec une fenêtre de gratuité distincte du butoir existant), montant annoncé au client avant sa décision puis figé sur la réservation. Le client peut enfin annuler une réservation confirmée, avec motif.

N.6 — exigences et chat sur la réservation confirmée. Nouvelle entité `ReservationRequirement` (agrégat gardé : demander → répondre → accepter / refuser avec motif / lever), écran agence dans le panneau de réservation, checklist avec dépôt de fichier dans l'espace client, et refus de conversion tant qu'il reste une exigence en attente. En parallèle, le fil de discussion n'est plus attaché à la seule location : `ChatMessage` porte désormais l'un ou l'autre des deux identifiants et les deux boîtes de réception listent les deux sortes de fils. Détail en N.6 ci-dessous.

N.1 — Expiration par défaut des papiers • livré

`Client` portait déjà `CINExpiryDate` , `PasseportExpiryDate` et `DrivingLicenceExpiryDate` (addendum A.2), et un permis périmé bloquait déjà la location. Ce qui manquait — le calcul depuis la date de délivrance — est en place, avec les durées réglables par agence.

À savoir : un permis délivré il y a douze ans dans une agence qui déclare dix ans de validité devient **périmé** et bloque la location, sauf acquittement explicite (`AcknowledgeExpiredDocuments`). C'est l'effet voulu — c'est précisément le dossier que l'assurance refuserait — mais il apparaîtra sur des fiches clients anciennes dès la première modification.

N.2 / N.3 — TTC et TVA

Déjà en place et volontairement sans interrupteur : le tarif journalier saisi est TTC, la facture redescend au HT (`TaxBreakdown.FromGross`), et le taux, le timbre et le matricule fiscal sont figés sur chaque facture à l'émission. Reste un point d'interface : les libellés de prix ne disent pas « TTC », ce qui est la seule raison pour laquelle un gérant peut croire l'inverse.

N.4 — Devises et taux · **décidé : conversion à l'affichage uniquement**

L'agence conserve **une** devise de facturation. Le marketplace affiche les prix convertis dans la devise choisie par le visiteur, via une table de taux (`ExchangeRate(From, To, Rate, AsOf)`) tenue à la plateforme. Factures, paiements, crédits et statistiques restent intégralement dans la devise de l'agence, et aucun montant stocké ne change de sens.

Écarté : la facturation multi-devise réelle (taux figé par document, écarts de change). Elle toucherait `Money` , les factures, les paiements et les statistiques pour un besoin qui est, à ce stade, un besoin d'affichage.

N.5 — Demande de réservation en attente + notification · livré

`NotificationKind.ReservationPending` rejoint la file de travail de l'agence, émis à la création d'une demande (agence et marketplace) et de nouveau par le balayage horaire à l'approche de l'expiration du délai de garde (`ReservationExpiryHours`). Côté client, la confirmation et le refus donnaient déjà lieu à un message ; c'était le signal **entrant** qui manquait.

N.6 — Réservation confirmée : exigences, paiement, chat · livré

Les exigences. Sur une réservation confirmée, l'agence déclare ce qu'elle attend avant la remise des clés — règlement (avec montant et mode attendu), caution, pièces à fournir, conditions à accepter, contrat à signer — sous forme de lignes `ReservationRequirement` . Le client répond depuis son espace (dépôt d'un fichier, ou acceptation), l'agence accepte, refuse **avec un motif que le client voit**, ou lève l'exigence. Renvoyer la liste la remplace : une exigence sans réponse est supprimée, une exigence déjà répondue est *levée* — le fichier du client et la trace de ce qui a été demandé ne disparaissent jamais.

La conversion est refusée tant qu'il reste quelque chose en attente, sauf `AcknowledgeUnmetRequirements` — même forme que l'acquiescement des papiers périmés, parce que l'agent qui convertit n'est souvent pas celui qui a posé les conditions.

L'argent reste un `Payment` : une exigence est la *demande*, jamais le grand livre. Le paiement demeure hors ligne, donc le §2.3 et l'Outbox (§4.6) restent hors périmètre.

Le chat. Un fil est désormais attaché à une **réservation confirmée** aussi bien qu'à une location :

`ChatMessage` porte `RentingId` ou `ReservationId` , `ChatSubjectKind` dit lequel, et `CanPostTo` a une surcharge par type (`NotYet` / `InProgress` d'un côté, `Confirmed` / `Paid` de l'autre). Une demande encore en attente ne s'ouvre pas aux messages : la réponse à une demande est « confirmer » ou « refuser », pas un message. Les deux boîtes de réception — celle de l'agence et celle du client — listent les deux sortes de fils ensemble, et la conversation d'une réservation **reste sur la réservation** après conversion.

N.7 — Fiabilité du client et frais d'annulation · livré

Un client qui annule souvent perd des points : 100 au départ, –15 par annulation et –15 de plus si elle était tardive. La note est **affichée à l'agence au moment de répondre à une demande**, avec les annulations qui l'ont produite — une note sans son motif ne se décide pas — et seulement quand il y a quelque chose à dire.

Deux principes d'équité y sont câblés :

- seule une annulation **faite par le client** compte. Une réservation annulée par l'agence, refusée, ou expirée faute de réponse ne lui coûte rien (`Reservation.CancelledByCustomer`) ;
- la note est **strictement par agence**. Ce qu'un client a fait ailleurs ne regarde pas cette agence — le même raisonnement qui garde `Client.IsFlagged` hors du marketplace. *Décision à arbitrer si vous voulez une réputation inter-agences : c'est un autre sujet, et il touche la vie privée d'une personne physique, pas la vitrine d'une entreprise.*

Les frais. Le `CancellationWindowHours` existant est un **butoir** : passé lui, on ne peut plus annuler du tout. Les frais avaient donc besoin d'une seconde borne, `CancellationFreeHours` : au-delà, l'annulation est gratuite ; entre les deux, elle est possible et coûte les frais (montant fixe ou pourcentage du prix, plafonné au prix). L'arithmétique vit dans un seul objet, `CancellationPolicy` , pour que le montant annoncé au client avant sa décision soit exactement celui qui lui est facturé.

Le client peut désormais annuler une réservation **confirmée**, avec motif — il ne le pouvait pas avant, alors que c'est précisément le cas où l'annulation coûte quelque chose. Le montant est figé sur la réservation à l'annulation.

N.8 — Annulation par l'agence · **décidé : pénalité de note, sans argent**

Une agence qui annule sans motif voit sa note de fiabilité baisser, et cette note est visible sur le marketplace. Le client peut ouvrir un **signalement**, arbitré par l'administrateur plateforme — ce qui rejoint le point « Signalements » déjà au §3.6 et le rend prioritaire.

Écarté : des frais d'annulation réellement payés par l'agence au client. Tant qu'aucun flux d'argent ne passe par la plateforme, une dette qu'aucun mécanisme ne recouvre est une ligne dans une table, pas une sanction.

N.9 — Chevauchement bloquant à la confirmation · **livré**

Le diagnostic d'origine était incomplet : une demande en attente **bloquait** déjà la voiture, si bien qu'une deuxième demande sur la même période était refusée dès la création — et la situation décrite (deux demandes, l'agence en annule une) ne pouvait pas se produire.

Arbitrage retenu : une demande en attente ne bloque plus rien, et c'est la confirmation qui tranche. Voir l'encadré « Ce qui a été livré » ci-dessus pour ce que cela a changé dans le code.

N.10 — Assistant d'ouverture d'agence

Un parcours en cinq étapes pour l'administrateur plateforme comme pour une future inscription en libre-service (§2.8) : détails de l'agence → modules activés → paramètres (facture, contrat, TVA) → voitures (saisie ou import) → publication sur le marketplace. La publication devient explicite : une agence en cours de configuration n'apparaît pas dans la recherche publique.

Ordre d'exécution retenu

~~N.9~~ → ~~N.1~~ → ~~N.5~~ → ~~N.6~~ → ~~N.7~~ → **N.8** (avec les signalements du §3.6) → N.4 → N.10, puis les libellés « TTC » de N.2.

N.6 est passé devant N.7 sur votre demande. Cinq points sont livrés (2026-09-02) ; la prochaine étape est N.8 — l'annulation par l'agence et les signalements, qui réutilisent la mécanique de note posée par N.7.